# Technical Companion:

Forecasting the 2026 FIFA World Cup: A Market-Calibrated Poisson–Elo Model and the Information Each Result Reveals

Avralt-Od Purevjav Jeronimo Luza

June 2026

**Abstract**

This document is the mathematical companion to the above paper. Every pipeline step is presented as the mathematics it implements, every modeling assumption is stated with its known limitations, and every formula is anchored to the exact code that executes it. It reads side-by-side with the paper; section numbers mirror the paper's stages 0–6 (Part I) and 7–12 (Part II).

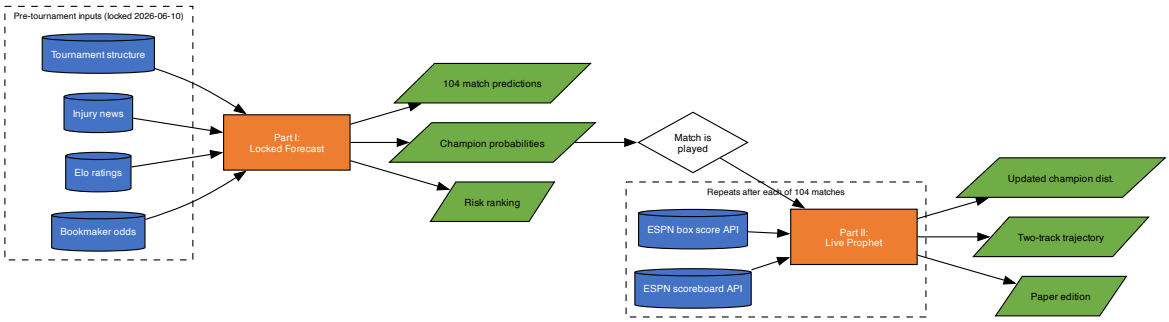



**Figure C1:** Master overview of the two-phase pipeline. Part I (locked forecast, Figure C2) runs once before the tournament; Part II (live Prophet, Figure C3) repeats after each of 104 matches.

## Contents

# §0 Notation and contract

Throughout, $i$ indexes teams and $R_i$ is team $i$'s Elo rating (a scalar strength, e.g. Spain 2165, Qatar 1423). A match between designated home side $H$ and away side $A$ produces a scoreline $(h, a) \in \mathbb{Z}^2_{\geq 0}$. The model's prediction is $(\hat{h}, \hat{a})$. Goal rates $\lambda_H, \lambda_A > 0$ are the expected goals of each side under a Poisson model. Outcome probabilities are the triple $(p_H, p_D, p_A)$ — home win, draw, away win — and $o_j$ denotes a decimal bookmaker odds quote for outcome $j \in \{H, D, A\}$. The pool score function is $S(\hat{h}, \hat{a};\ h, a) \in \{0, 1, 2, 3\}$ (defined in §). In Part II, $p_t$ denotes the champion probability distribution over all 48 teams after conditioning on the first $t$ played games, and $d_i$ is team $i$'s learned rating drift.

Two contracts hold throughout this document:

1. **Anchor contract.** Every displayed equation is followed by an anchor naming the code that implements it — `scripts/file.py:function` — with the constants as locked before kickoff (2026-06-10). The formulas are extracted from the code, not from an idealized model.

2. **Honesty rule.** Where the code is a pragmatic shortcut rather than clean statistics (an independence assumption, a proportional vig split, a memoized grid), the note says so explicitly rather than dressing it up.

# §0.5 Complete variable inventory

### Tier 1 — Pre-tournament inputs (locked 2026-06-10)

**Table C1:** Pre-tournament inputs. All files are committed before kickoff (tag `prereg-2026`) and never re-read from the outside world.

| Symbol | What | How measured | Values / range | File |
|---|---|---|---|---|
| $o_H, o_D, o_A$ | Bookmaker decimal odds | Hand-collected Jun 2–7 from bet365, FanDuel, DraftKings, Betfair, Kalshi/Polymarket; one source per fixture, recorded in `source` field | e.g. Mexico–S.Africa: 1.40 / 4.60 / 8.00 | `data/odds_AD.json`, `odds_EH.json`, `odds_IL.json` |
| $R_i$ | Elo rating, 48 teams | Read from ClubElo/EloRatings.net, Jun 5–7 | Spain 2165 … Qatar 1423 | `data/elo_outright_news.jso` |
| $\delta_i^{\text{inj}}$ | Injury Elo deduction | Hand-set scalar per confirmed absence | NED −23, BRA −20, JPN −10, CRO −10, ESP −5 | `data/elo_outright_news.jso` |
| $\delta_i^{\text{mkt}}$ | Market recalibration | Prediction-market implied 140-pt Portugal–Colombia gap vs. Elo 7-pt → split | POR +66, COL −67 | `scripts/condition.py` |
| $\delta_{\text{host}}$ | Host venue bonus | Fixed rule; decays in late rounds | +40 R32/R16; +20 QF onward | `scripts/condition.py` |
| GROUP_FIXTURES | 72 group fixtures | Static table (row, group, home, away) | — | `scripts/fixtures.py` |
| KNOCKOUT | 32-slot bracket wiring | Directed graph, FIFA R32/R16/QF/SF/Final | — | `scripts/fixtures.py` |

## Tier 2 — Intermediate computed values

**Table C2:** Intermediate values derived during pipeline execution.

| Symbol | What | Formula / source | Range |
|---|---|---|---|
| $p_H, p_D, p_A$ | Fair outcome probabilities | $p_j = (1/o_j)/\sum_k (1/o_k)$; or Elo synthesis for odds-free matches | $[0, 1]$, sum = 1 |
| $\lambda_H, \lambda_A$ | Expected goal rates | `fit_rates`: grid search minimise $\sum(\tilde{p}_j - p_j)^2$, step 0.05, total goals $\in [1.6, 3.4]$ | 0.10–3.50 each |
| $P(h, a)$ | Scoreline distribution | $\text{Pois}(h; \lambda_H) \cdot \text{Pois}(a; \lambda_A)$, $(h, a) \in \{0, \ldots, 8\}^2$ | 81 values, sums to $\approx 1$ |
| $\hat{h}, \hat{a}$ | Match prediction | $\arg\max \mathbb{E}[S]$ over $9 \times 9$ grid (EV rule); or modal-conditional; or realistic readout | integers 0–8 |
| $\mathbb{E}[S(\hat{h}, \hat{a})]$ | Expected pool points | $p_{\text{exact}} + p_{\text{result+GD}} + p_{\text{result}}$ (exact, no simulation) | $[0, 3]$ |
| $\lambda_{\text{obs}}$ | Observed performance (proxy xG) | $0.3256 \times$ SoT; calibrated OLS on 236 WC team-matches | $\geq 0$ |
| $\lambda_{\text{exp}}$ | Expected performance | Elo synthesis + `fit_rates` at current $R_i^{\text{eff}}$; memoised on integer Elo diff | 0.10–3.50 |
| $s$ | Net surprise (home) | $(\lambda^H_{\text{obs}} - \lambda^H_{\text{exp}}) - (\lambda^A_{\text{obs}} - \lambda^A_{\text{exp}})$; zero-sum by construction | $\mathbb{R}$ |
| $d_i$ | Per-team rating drift | $d \leftarrow 0.95\, d + \text{clip}(50\, s, \pm 75)$; initialised 0 at kickoff | $\approx \pm 214$ max over group stage |
| $R_i^{\text{eff}}$ | Effective rating | $R_i + \delta_i^{\text{inj}} + \delta_i^{\text{host}} + \delta_i^{\text{mkt}} + d_i$ | — |
| $p_t$ | Champion distribution at time $t$ | Monte Carlo counter over $N$ simulations | $[0, 1]$ over 48 teams, sum = 1 |
| $D_{\text{KL}}(p_t \Vert p_{t-1})$ | Per-game information (bits) | $\sum_i p_t(i) \log_2[p_t(i)/\max(p_{t-1}(i), \varepsilon)]$, $\varepsilon = 10^{-12}$ | $\geq 0$ bits |

## Tier 3 — Hyperparameters and tuning constants (locked pre-tournament)

**Table C3:** All constants locked at kickoff (2026-06-10).

| Constant | Value | Rationale |
|---|---|---|
| `MAX_G` | 8 | Poisson tail $< 2.4 \times 10^{-4}$ at $\lambda = 2$ |
| $N_{\text{pre}}$ | 200,000 | MC SE $\approx \pm 0.1$ pp at champion scale |
| $N_{\text{cond}}$ | 50,000 | Per-track post-match re-simulation |
| $N_{\text{replay}}$ | 4,000 | Per-snapshot in 2022 replay validation |
| seed | 2026 | Fixed for reproducibility |
| $\lambda$ grid step | 0.05 | Balances inversion precision vs. compute |
| total goals band | $[1.6, 3.4]$ | WC group games average $\approx$ 2.5–2.7 total goals |
| $k$ (learning gain) | 50 | Midpoint of 2022 optimum (40) and 2018 optimum (60) |
| $\gamma$ (decay) | 0.95 | Drift half-life $\approx$ 13.5 matches |
| $B$ (drift bound) | 75 | Hard clip on per-match step |
| $\varepsilon$ (KL floor) | $10^{-12}$ | Prevents $\log 0$ in divergence |
| host bonus early | +40 | R32 and R16 rounds |
| host bonus late | +20 | QF, SF, Final |
| draw share base | 0.30 | Decays as $e^{-\lvert d\rvert/700}$ with Elo gap |
| pen threshold | 0.25 | Winner's rate edge $< 0.25$ goals $\Rightarrow$ penalties pick |

## §0.6 Match lifecycle variables

### §0.6a — Before-game state (entering match $m$)

**Table C4:** Variables the model knows entering match $m$, and whether they update after each prior match.

| Symbol | What | How determined | Updated each match? |
|---|---|---|---|
| $R_i$ | Baseline Elo | Locked Jun 10, from `elo_outright_news.json` | No |
| $\delta_i^{\text{inj}}$ | Injury deduction | Hand-set pre-tournament | No |
| $\delta_i^{\text{host}}$ | Host venue bonus | Fixed rule applied at sim time | No |
| $\delta_i^{\text{mkt}}$ | Market recalibration | Set once from prediction-market gap | No |
| $d_i$ | Accumulated drift (Track B; Track A: always 0) | §9 update after each prior match | Yes — every match |
| $R_i^{\text{eff}} = R_i + \delta^{\text{inj}} + \delta^{\text{host}} + \delta^{\text{mkt}} + d_i$ | Effective rating entering $m$ | Composition of above | Yes (via $d_i$) |
| $o_H, o_D, o_A$ | Bookmaker odds for match $m$ | Hand-collected Jun 2–7 (group matches only) | No |
| $p_H, p_D, p_A$ | Fair outcome probabilities | Normalised odds; or Elo synthesis (KO / odds-free) | No |
| $\lambda_H, \lambda_A$ | Expected goal rates for match $m$ | `fit_rates` from $(p_H, p_D, p_A)$ | No (group) / derived from $R_i^{\text{eff}}$ (KO) |
| $P(h, a)$ | Scoreline distribution, 81 values | $\text{Pois}(h; \lambda_H) \cdot \text{Pois}(a; \lambda_A)$ | No |
| $\hat{h}, \hat{a}$ | Published prediction for match $m$ | $\arg\max \mathbb{E}[S]$ or realistic readout | No |
| $p_{t-1}$ | Champion distribution entering $m$ | Previous match's re-simulation output | Yes — previous match |
| $\text{Piv}(m)$ | Pre-match pivotality (expected KL, bits) | $\sum_o w_o D_{\text{KL}}(p^{\text{post}(o)} \| p^{\text{pre}})$, $N$ = 5,000 per fork | Computed fresh |

### §0.6b — After-game variables (once the whistle blows)

**Table C5:** Variables that arrive or are computed after match $m$ is completed.

| Symbol | What | Source | Available when |
|---|---|---|---|
| $h, a$ | Final score | ESPN scoreboard API → `fetch_results.py` | ~5 min after FT |
| $r = \text{sgn}(h - a)$ | Realized result $\in \{+1, 0, -1\}$ | Derived | Same |
| $S(\hat{h}, \hat{a};\, h, a)$ | Pool points earned $\in \{0, 1, 2, 3\}$ | `scoring.py:score_match` | Same |
| $\text{SoT}_H, \text{SoT}_A$ | Shots on target | ESPN summary API → `fetch_stats_espn.py` | ~15 min after FT |
| $\text{otherShots}_H, \text{otherShots}_A$ | Off-target + blocked shots | Same | Same |
| $\text{poss}_H, \text{poss}_A$ | Possession % | Same (scraped; $\beta_2 = 0$, not used in proxy) | Same |
| $\lambda^H_{\text{obs}}, \lambda^A_{\text{obs}}$ | Observed performance (proxy xG) | $0.3256 \times$ SoT; from shot stats | After stats arrive |
| $\lambda^H_{\text{exp}}, \lambda^A_{\text{exp}}$ | Expected performance | Elo synthesis + `fit_rates` at pre-match $R_i^{\text{eff}}$ | Pre-computed |
| $s_H$ | Net surprise | $(\lambda^H_{\text{obs}} - \lambda^H_{\text{exp}}) - (\lambda^A_{\text{obs}} - \lambda^A_{\text{exp}})$; zero-sum | Derived |
| $d_i^{\text{new}}$ (Track B) | Updated drift | $0.95\, d_i + \text{clip}(50\, s_i, \pm 75)$ | Derived from $s$ |
| $p_t^{\text{frozen}}$ | Champion dist., Track A | Re-sim $N = 50{,}000$, results pinned, ratings fixed | ~min after FT |
| $p_t^{\text{learn}}$ | Champion dist., Track B | Re-sim $N = 50{,}000$, results pinned, $R_i^{\text{eff}}$ updated | Same |
| $D_{\text{KL}}(p_t^{\text{frozen}} \Vert p_{t-1}^{\text{frozen}})$ | Result-channel information (bits) | Scoreline conditioning alone | Derived |
| $D_{\text{KL}}(p_t^{\text{learn}} \Vert p_t^{\text{frozen}})$ | Performance-channel information (bits) | Gap between tracks after match $m$ | Derived |

# Part I
# The Locked Forecast

## 1 From information to numbers

Everything downstream is arithmetic on numbers, so the first modeling step is *measurement*: turning four kinds of worldly information into typed numerical objects.

**(a) Bookmaker odds → probability triples.** Each group fixture is one JSON record carrying decimal odds and normalised fair probabilities: $(o_H, o_D, o_A) \mapsto (p_H, p_D, p_A)$ with $p_j \propto 1/o_j$ (the normalisation is §2's subject). Odds were hand-collected June 2–7, 2026 from bet365, FanDuel, DraftKings, Betfair, and Kalshi/Polymarket composites — one source per fixture, recorded in a `source` field, not averaged across books. Coverage: 55 of 72 group fixtures. The loader merges three files (`odds_AD.json`, `odds_EH.json`,

`odds_IL.json`), keys by canonicalised team pair, and defensively renormalises $p_j \leftarrow p_j/(p_H + p_D + p_A)$. The 17 matchday-3 fixtures with no published odds at collection time receive an Elo estimate instead (flagged in `source`).

Anchor. `scripts/predict_groups.py:load_odds + main --- pair-keyed merge, orientation flip, renormalisation.`

**(b) Elo table → ratings $R_i$.** One integer per team for all 48 teams (Spain 2165 down to Qatar 1423), read from `data/elo_outright_news.json`. The effective rating entering any match sums four corrections:

$$R_i^{\text{eff}} = R_i + \delta_i^{\text{inj}} + \delta_i^{\text{host}} + \delta_i^{\text{mkt}} + d_i. \tag{1}$$

In Part I the drift is initialised to zero ($d_i = 0$) so the locked forecast uses $R_i + \delta_i^{\text{inj}} + \delta_i^{\text{host}} + \delta_i^{\text{mkt}}$. The market recalibration $\delta_i^{\text{mkt}}$ and drift $d_i$ are defined in Tables C1 and C2 respectively.

**(c) Team news → additive rating penalties.** Qualitative injury reports are encoded as fixed Elo deductions $\delta_i^{\text{inj}}$, set once at collection time:

| Team | $\delta^{\text{inj}}$ | Reason |
|---|---|---|
| Netherlands | −23 | Xavi Simons out |
| Brazil | −20 | Rodrygo out |
| Japan | −10 | Mitoma out |
| Croatia | −10 | squad fitness |
| Spain | −5 | Fermín López out (metatarsal) |

> *Honesty note.* This is the bluntest measurement in the pipeline — a hand-set scalar per absence, not a player-value model.

**(d) Tournament structure → fixture list + bracket graph.** The 72 group fixtures are a static table (`row, group, home, away`); the knockout is a directed graph encoded as the sheet's slot layout. Two auxiliary encodings matter: a deterministic alias map (`"Türkiye"`→`"Turkey"`, `"Korea Republic"`→`"South Korea"`, …) and a fixed bijection between sheet rows 4–75 and FIFA's official group-match numbers 1–72, cross-checked by a unit test.

Anchor. `scripts/fixtures.py:GROUP_FIXTURES/KNOCKOUT/ALIASES/canon/ROW_MATCH (numbering guarded by tests/test_match_numbering.py).`

## 2 Clean the odds

A decimal quote $o_j$ implies probability $1/o_j$, but a book's implied probabilities sum to more than one. The excess is the *overround* (vig),

$$V = \sum_{j \in \{H,D,A\}} \frac{1}{o_j} - 1 > 0. \tag{2}$$

The model strips it by *proportional normalisation*:

$$p_j = \frac{1/o_j}{\sum_{k \in \{H,D,A\}} 1/o_k}, \qquad j \in \{H, D, A\}. \tag{3}$$

**Worked example.** Mexico–South Africa at $(1.40, 4.60, 8.00)$ implies $(0.714, 0.217, 0.125)$, $V = 5.6\%$, normalising to $(0.676, 0.206, 0.118)$.

> *Honesty note.* Proportional de-vigging spreads the vig across outcomes in proportion to their implied probability. Books are known to shade longshots more than favourites (the favourite–longshot bias), which Shin-type methods would correct. The pipeline knowingly uses the proportional map: the bias is second-order at World Cup group-match prices, and the downstream pick rule is insensitive to probability perturbations of that size.

Anchor. `data/odds_*.json (fields odds_* and p_*); scripts/predict_groups.py:main --- ph,pd,pa = ph/s, pd/s, pa/s.`

# 3 The Poisson scoreline model

## The model

Goals are independent Poisson counts:

$$P(h, a) = \text{Pois}(h; \lambda_H)\,\text{Pois}(a; \lambda_A) = e^{-\lambda_H}\frac{\lambda_H^h}{h!} \cdot e^{-\lambda_A}\frac{\lambda_A^a}{a!}, \tag{4}$$

truncated to $(h, a) \in \{0, \ldots, 8\}^2$ everywhere.

Anchor. `scripts/poisson_model.py:pois (MAX_G = 8).`

> *Honesty note.* Real scorelines exhibit mild dependence between the two counts (bivariate-Poisson and Dixon–Coles corrections exist for exactly this); the pipeline uses independence throughout.

## The inverse problem

The market gives $(p_H, p_D, p_A)$; the model needs rates. Outcome probabilities under the model are the grid sums $\tilde{p}_H = \sum_{h>a} P(h, a)$, $\tilde{p}_D = \sum_{h=a} P(h, a)$, $\tilde{p}_A = \sum_{h<a} P(h, a)$, and `fit_rates` inverts the map by constrained least squares on a lattice:

$$(\hat{\lambda}_H, \hat{\lambda}_A) = \underset{\substack{\lambda_H, \lambda_A \in \{0.10, 0.15, \ldots, 3.50\} \\ 1.6 \le \lambda_H + \lambda_A \le 3.4}}{\arg\min} \sum_{j \in \{H, D, A\}} \big(\tilde{p}_j(\lambda_H, \lambda_A) - p_j\big)^2. \tag{5}$$

The lattice step is 0.05; the total-goals band $[1.6, 3.4]$ encodes the prior that World Cup group games average roughly 2.5–2.7 total goals.

Anchor. `scripts/poisson_model.py:outcome_probs/fit_rates --- steps = 0.05k, k = 2..70; outcome_probs is lru_cached so the` $69 \times 69$ `grid search costs one pass.`

## The EV-optimal pick rule

The model's pick is the expected-points maximiser:

$$(\hat{h}, \hat{a}) = \underset{(\hat{h}, \hat{a}) \in \{0, \ldots, 8\}^2}{\arg\max} \; \mathbb{E}\big[S(\hat{h}, \hat{a};\, h, a)\big], \tag{6}$$

found by brute force over all 81 candidate scorelines.

Anchor. `scripts/ev321.py:best_pick --- exhaustive search over the` $9 \times 9$ `grid.`

### The modal-conditional rule (published picks)

The most likely exact score conditional on the most likely outcome:

$$(\hat{h}, \hat{a}) = \underset{(h,a):\,\mathrm{sgn}(h-a)=r^*}{\arg\max} P(h,a), \qquad r^* = \arg\max_j p_j. \tag{7}$$

Matches rule (6) on 71 of 72 group fixtures.

Anchor. `scripts/poisson_model.py:modal_score.`

### Knockout Elo synthesis

Undrawn knockout ties have no odds, so $(p_H, p_D, p_A)$ is synthesised from effective Elo ratings ($d = R_H^{\text{eff}} - R_A^{\text{eff}}$):

$$e = \frac{1}{1 + 10^{-d/400}}, \tag{8}$$

$$p_D = 0.30\, e^{-|d|/700}, \tag{9}$$

$$p_H = \max\!\left(0.01,\ e - \tfrac{p_D}{2}\right), \tag{10}$$

$$p_A = \max(0.01,\ 1 - p_H - p_D), \tag{11}$$

renormalised to sum to one, then fed through `fit_rates`.

Anchor. `scripts/realistic_scores.py:main (KO branch); scripts/predict_knockout.py:probs.`

## 4 Monte Carlo simulation

Quantities like "P(Spain champion)" have no closed form. They are expectations over tournament realisations $\omega$:

$$\theta = \mathbb{E}[\mathbf{1}\{\text{event}(\omega)\}] \approx \hat{\theta}_N = \frac{1}{N}\sum_{n=1}^{N} \mathbf{1}\{\text{event}(\omega_n)\}. \tag{12}$$

The locked run used $N = 200{,}000$, so the Monte Carlo standard error at a champion-scale probability is

$$\mathrm{SE}(\hat{\theta}) = \sqrt{\frac{\theta(1-\theta)}{N}} \approx \sqrt{\frac{0.27 \times 0.73}{200{,}000}} \approx 0.001, \tag{13}$$

i.e. about ±0.1 percentage points.

Anchor. `scripts/simulate.py --- N_SIMS = 200,000, random.seed(2026).`

**One tournament draw.** Each $\omega_n$ is generated in four stages:

1. **Group scorelines.** Every group match draws from $\mathrm{Pois}(\lambda_H) \wedge 8$ and $\mathrm{Pois}(\lambda_A) \wedge 8$ independently by inverse-CDF sampling.
2. **Standings.** Group tables sort descending by $(\text{Pts}, \text{GD}, \text{GF}, U)$, $U \sim \mathrm{Unif}(0,1)$.
3. **Third-place slotting.** Top 8 of 12 third-placed teams advance; placed by bipartite matching against FIFA's allocation constraints.

4. **Knockouts.** Each tie is a Bernoulli draw from the Elo logistic

$$P(A \text{ beats } B) = \frac{1}{1 + 10^{-(R_A^{\text{eff}} - R_B^{\text{eff}})/400}}, \tag{14}$$

which is read as the probability of advancing by any means (extra time and shootout included).

ANCHOR. `scripts/sim_tournament.py:group_standings/simulate_knockout/simulate.`

# 5 Slot emergence and pivotality

## Slot emergence

A knockout pick at bracket slot $m$ scores only if the named team occupies the slot *and* wins it. For team $t$ and slot $m$, define from the §4 simulation

$$E_m(t) = P(t \text{ wins match } m) = P(t \text{ reaches } m) \cdot P(t \text{ wins} \mid t \text{ reaches } m). \tag{15}$$

The pick rule is $\hat{t}_m = \arg\max_t \; E_m(t)$.

ANCHOR. `scripts/simulate.py (second pass, slot_win counters); scripts/slot_value.py:simulate/en`

## Pivotality

For an upcoming match $m$ with pre-match champion distribution $p^{\text{pre}}$, fork the match across its possible outcomes $o$, recompute the conditioned champion distribution $p^{\text{post}(o)}$, and average the information distance:

$$\text{Piv}(m) = \sum_o w_o \, D_{\text{KL}}\big(p^{\text{post}(o)} \,\big\|\, p^{\text{pre}}\big) \quad \text{bits}, \tag{16}$$

with $\varepsilon = 10^{-6}$ added to both distributions before the $\log_2$ ratio. This is the ex-ante counterpart of the realised per-game KL of §11.

ANCHOR. `scripts/pivotality.py:kl/main --- N = 5,000 per fork, seed 2026, output data/pivotality.`

# 6 The machine learning risk layer

The simulator is also a data generator. Each simulated tournament yields one labelled example of the question "given which group calls came true, how many knockout picks scored?":

- **Features** $x \in \{0, 1\}^{24}$: for each group $g \in \{A, \ldots, L\}$, two indicators — winner correct and runner-up correct.
- **Target** $y \in \{0, \ldots, 31\}$: number of the sheet's 31 knockout advancers that advance in that simulation.

Dataset: $N = 40{,}000$ draws (seed 7), split 75/25 train/test. Two regressors $\hat{f}(x) \approx \mathbb{E}[y \mid x]$:

| Model | Hyperparameters |
|---|---|
| Random Forest | 300 trees, max depth 12, min leaf 20 |
| XGBoost | 400 trees, max depth 4, learning rate 0.05, subsample 0.8, colsample 0.8 |

SHAP values decompose each prediction additively, $\hat{f}(x) = \phi_0 + \sum_{i=1}^{24} \phi_i(x)$; the report ranks features by mean $|\phi_i|$ and quantifies the directional effect $\mathbb{E}[\phi_i \mid x_i = 1]$.

*Honesty note.* The regression is trained on the simulator's own output: it is a sensitivity analysis of the model, not an independent second forecast. It changes no pick.

ANCHOR. `scripts/ml_risk_xgb.py --- shap.TreeExplainer, output data/ml_risk_xgb.json.`

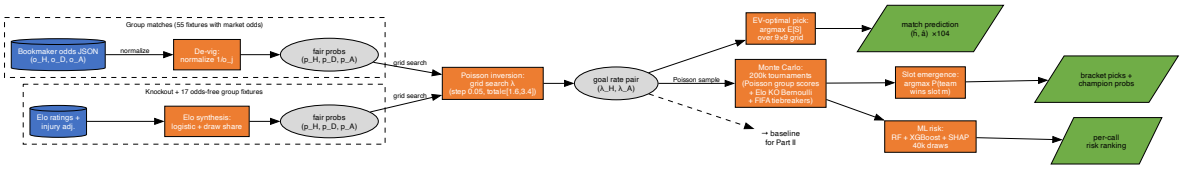



**Figure C2:** Part I locked forecast pipeline in detail. Two input lanes (market odds for group matches; Elo synthesis for knockout matches and the 17 odds-free group fixtures) merge at the fitted goal-rate pair $(\lambda_H, \lambda_A)$, which feeds both the EV-optimal match prediction and the 200k-tournament Monte Carlo.

# Part II
# The Live Prophet

## 7 Measurement: from match events to $\lambda_{\text{obs}}$

Real xG is not freely available for World Cups. The free signal that survives is the box score: shots on target and other shots. Define the observed performance rate of a team as

$$\hat{\lambda}_{\text{obs}} = \max\big(0,\ \beta_1 \cdot \text{SoT} + \beta_2 \cdot \text{otherShots}\big), \qquad \beta_1 = 0.3256, \quad \beta_2 = 0, \tag{17}$$

where otherShots = off-target + blocked. The calibrated $\beta_2$ is exactly zero: each shot on target is worth about a third of an expected goal; off-target and blocked shots carry no incremental information once shots on target are counted.

**Calibration.** The coefficients are a one-time least-squares fit of realised goals on shot counts over the 2018 and 2022 World Cups ($n = 236$ team-matches of free ESPN box-score data):

$$(\beta_1, \beta_2) = \arg\min_{\beta \geq 0} \sum_{i=1}^{236} \big(\text{goals}_i - \beta_1\, \text{SoT}_i - \beta_2\, \text{otherShots}_i\big)^2, \tag{18}$$

OLS through the origin, clipped to non-negative coefficients.

ANCHOR. `scripts/performance.py:proxy_xg; coefficients in data/proxy_coef.json --- sot: 0.32557, other: 0.0.`

**Fallback.** If a real xG number is available for a match, it takes precedence, with the source recorded per match so the two never mix silently.

ANCHOR. `scripts/performance.py:compute_lambda_obs --- source: 'real' | 'proxy'.`

# 8 The expectation model: $\lambda_{\text{exp}}$ from ratings

Given two teams' current ratings, the expected performance rates are produced by the same chain Part I uses for odds-free matchups — one composition of three maps applied to the rating difference $d = R_{\text{home}} - R_{\text{away}}$:

$$d \xrightarrow[\text{Elo logistic + draw}]{\text{(i)}} (p_H, p_D, p_A) \xrightarrow[\texttt{fit\_rates}]{\text{(ii)}} (\lambda_{\text{exp}}^{\text{home}}, \lambda_{\text{exp}}^{\text{away}}), \tag{19}$$

where step (i) is the synthesis of equations (8)–(11) and step (ii) is the lattice least-squares inversion of equation (5). Nothing new is fitted here: the Prophet inherits Part I's match model wholesale.

**Numerics.** $\lambda_{\text{exp}}$ depends only on $d$ rounded to the nearest integer Elo point, and `_lambda_for_diff` is memoised on that integer. A half-point rating change moves the win expectancy by under $10^{-3}$, far below the lattice resolution (0.05 goals) of the inversion it feeds, so the cache is lossless in effect.

Anchor. `scripts/learn.py:_lambda_for_diff/lambda_expected.`

# 9 The learning update: net surprise and regularized drift

## Net surprise

After a match, each team's performance is compared to expectation on both sides of the ball and netted:

$$s_{\text{home}} = \underbrace{\left(\lambda_{\text{obs}}^{H} - \lambda_{\text{exp}}^{H}\right)}_{\text{attack surplus}} - \underbrace{\left(\lambda_{\text{obs}}^{A} - \lambda_{\text{exp}}^{A}\right)}_{\text{defence leak}}, \qquad s_{\text{away}} = -s_{\text{home}}. \tag{20}$$

The construction is zero-sum by definition: one team's positive surprise is exactly the other's negative, so learning redistributes strength rather than inflating it.

Anchor. `scripts/learn.py:net_surprise and LearningTrack.apply_match.`

## The drift update

Each team carries an accumulated drift $d_i$ (initialised to 0) on top of its frozen baseline, $R_i^{\text{learn}} = R_i + d_i$. One match updates it as

$$d_i \leftarrow \gamma\, d_i + \text{clip}(k\, s_i,\ -B,\ +B), \qquad k = 50, \quad \gamma = 0.95, \quad B = 75. \tag{21}$$

The clip binds the per-match *step* $k s_i$, not the accumulated drift. Setting $k = 0$ makes the step vanish and the drift stay at zero: the frozen track is exactly the learning track at zero gain, which is what makes the A/B comparison clean.

Anchor. `scripts/learn.py:update_drift --- step = max(-bound, min(bound, k*surprise)); return decay*drift + step; defaults in LearningTrack.__init__ (k=50.0, decay=0.95, bound=75.0).`

# 10 Re-simulation and the two-track trajectory

After $t$ games have been played, the forecast is the conditional distribution

$$p_t(\text{team}) = P\big(\text{team wins} \,\big|\, \text{results}_{1..t},\ \text{ratings}_t\big), \tag{22}$$

estimated by Monte Carlo with played games pinned and the future sampled. The two tracks differ *only* in $\text{ratings}_t$: the frozen track always passes the locked baseline $R_i$; the learning track passes $R_i + d_{i,t}$ with the drift of §9.

**The replay loop.** `run_replay` steps an ordered game list. For each finished game it (i) pins the result, (ii) applies the drift update from the game's $\lambda_{\text{obs}}$, and (iii) re-simulates the whole tournament twice — once per track — recording a snapshot $(p_t^{\text{frozen}}, p_t^{\text{learn}})$. The trajectory is

$$\text{trajectory} = \left\{ \left( p_t^{\text{frozen}}, p_t^{\text{learn}} \right) \right\}_{t=0}^{T}, \qquad \text{headline} = \text{the gap between the two paths.} \tag{23}$$

Anchor. `scripts/replay.py:run_replay --- N = 4,000 per snapshot, seed 2026.`

## 11 Information accounting: KL divergence in bits

Each snapshot records how far the new champion distribution moved from the previous one, in bits:

$$D_{\text{KL}}\left(p_t \,\|\, p_{t-1}\right) = \sum_{i:\, p_t(i)>0} p_t(i)\, \log_2 \frac{p_t(i)}{\max(p_{t-1}(i), \varepsilon)}, \qquad \varepsilon = 10^{-12}. \tag{24}$$

Base 2 makes the unit **bits**: one bit is the information in one fair coin flip about the championship. This is the *realised* counterpart of §5's pivotality: pivotality is the expected KL before kickoff; `kl_from_prev` is what the match actually delivered.

**Channel decomposition.**

- **Result channel**: the frozen track's own per-game divergence, summed over a window — what conditioning on scorelines alone teaches.
- **Performance channel**: $D_{\text{KL}}(p_T^{\text{learn}} \| p_T^{\text{frozen}})$ — what reading the shot counts added beyond the scorelines.

Across the 48 group games of the 2022 replay the per-game result and performance informations correlate at $-0.09$ — essentially orthogonal. That near-orthogonality is the empirical licence for the learning track's existence.

Anchor. `scripts/snapshot.py:kl_divergence; attached per snapshot as kl_from_prev in scripts/replay.py:_snapshot.`

## 12 How we know it works: validation

The learning claim was stress-tested against the two complete free-data tournaments available, 2018 and 2022. The evaluation metric is **finalist lift**: replay a tournament's group stage, then compare how much champion-probability mass each track's end-of-groups forecast assigns to the two teams that actually reached that final,

$$\text{lift} = \Big( \sum_{t \in \text{finalists}} p_T^{\text{learn}}(t) \Big) - \Big( \sum_{t \in \text{finalists}} p_T^{\text{frozen}}(t) \Big) \quad \text{percentage points.} \tag{25}$$

**(1) Leave-one-tournament-out cross-validation.** Tune $k$ on one World Cup, test on the other. Train-2018 fixes $k = 60$, tested on 2022; train-2022 fixes $k = 40$, tested on 2018. Learning helps out-of-sample in both directions.

Anchor. `scripts/heldout_validation.py part (1) --- N = 4,000, seed 7.`

**(2) Placebo control.** Shuffle the $\lambda_{\text{obs}}$ values across the tournament's games (4 independent shuffles, seeds 0–3) and re-run at the locked $k = 50$. The real-xG lift collapses to negative territory under shuffling, confirming the proxy carries real per-team information.

ANCHOR. `scripts/heldout_validation.py part (2) --- rng.shuffle(lams)`, $N = 3{,}000$ `per run.`

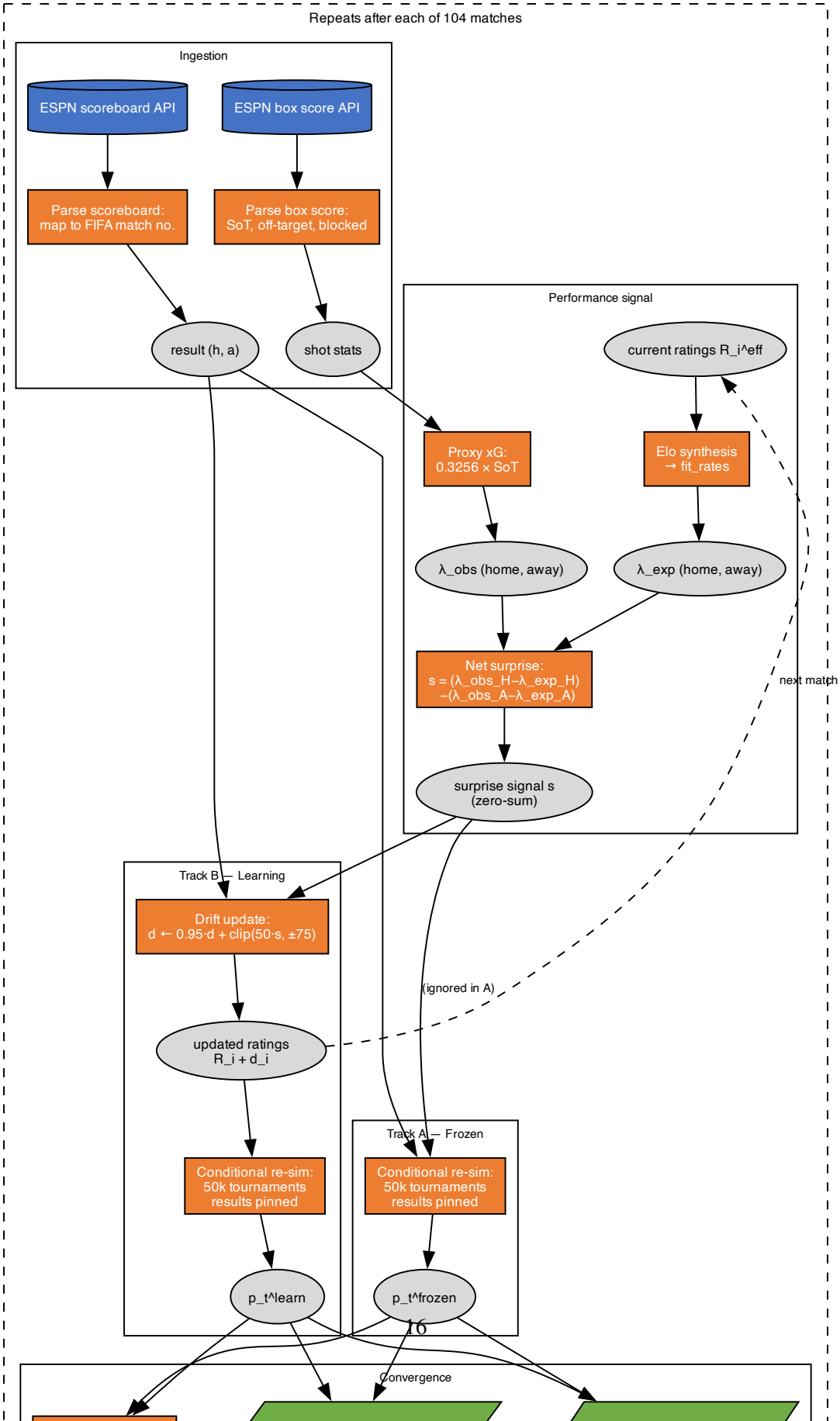
**(3) Proxy stability.** The $\beta_1$ coefficient refit on each tournament alone brackets the locked value (0.326); the cross-tournament goal RMSE beats the naive baseline (predict every team-match with the tournament mean goals) in both tournaments.

ANCHOR. `scripts/heldout_validation.py part (3) --- per-tournament refit` $c = \sum s_i g_i / \sum s_i^2$ `(OLS through the origin), RMSE comparisons.`

**The $k$-sweep.** Gain swept over $k \in \{0, 20, 40, 60, 80, 120, 160, 220, 300\}$ on full two-track replays of both tournaments. The curve is single-peaked in both years — learning helps, then over-reacts. Optima: $k = 40$ (2022), $k = 60$ (2018); locked default: midpoint $k = 50$.

ANCHOR. `scripts/sweep_k.py (`$N = 3{,}000$`, seed 2022), scripts/run_2018_ksweep.py, figure paper/figs/fig_ksweep_2tourn.pdf.`

**Limitations.** The sample is $n = 2$ tournaments; every number above carries that caveat. The validation shows the learning signal is real and correctly signed out-of-sample — it does not pin the optimal gain to better than "somewhere around 50," and the study is framed as exploratory in the paper accordingly. The 2026 live run is the genuine test.


Repeats after each of 104 matches
Ingestion
ESPN scoreboard API
ESPN box score API
Parse scoreboard: map to FIFA match no.
Parse box score: SoT, off-target, blocked
result (h, a)
shot stats
Performance signal
current ratings R_i^eff
Proxy xG: 0.3256 × SoT
Elo synthesis → fit_rates
λ_obs (home, away)
λ_exp (home, away)
Net surprise: s = (λ_obs_H−λ_exp_H) −(λ_obs_A−λ_exp_A)
surprise signal s (zero-sum)
next match
Track B — Learning
Drift update: d ← 0.95·d + clip(50·s, ±75)
(ignored in A)
updated ratings R_i + d_i
Track A — Frozen
Conditional re-sim: 50k tournaments results pinned
Conditional re-sim: 50k tournaments results pinned
p_t^learn
p_t^frozen
Convergence